

ATIVIDADE AULA 5

Prompt:

Contexto: Estou desenvolvendo o back-end de uma plataforma de e-commerce e preciso da estrutura inicial do banco de dados relacional.

Papel: Atue como um Arquiteto de Dados Sênior especializado em PostgreSQL.

Tarefa: Desenvolva um script SQL completo e funcional para as principais tabelas do sistema.

Requisitos Técnicos:

1. Crie as tabelas: `Clientes`, `Pedidos` e `Itens_Pedido`.
2. Implemente integridade referencial rigorosa usando chaves primárias (PK) e chaves estrangeiras (FK).
3. Use tipos de dados otimizados (ex: `SERIAL`, `NUMERIC`, `TIMESTAMP`).
4. Inclua restrições (`NOT NULL`, `UNIQUE`, `CHECK`).

Instrução de Raciocínio (CoT): Antes de apresentar o código, explique passo a passo a lógica da modelagem, justificando a escolha dos relacionamentos e como a integridade dos dados será garantida.

Formato de Saída:

1. Explicação do raciocínio em tópicos.
2. Script SQL em um único bloco de código Markdown.

Resultado:

Raciocínio Arquitetural (Chain of Thought)

Modelo de Dados: 3 tabelas relacionadas 1:N

- `Clientes (1) ← → (N) Pedidos ← → (N) Itens_Pedido`

Decisões Técnicas:

- Chaves Primárias: `BIGSERIAL` para performance
- Chaves Estrangeiras: `CASCADE` (cliente deletado = pedidos deletados), `RESTRICT` (não deleta pedido com itens)
- Valores Monetários: `DECIMAL(10,2)` para precisão

- Validação: CHECK constraints + tipos específicos
 - Performance: Índices em FKs e campos de busca
-

Script SQL

SQL

```
-- TABELA: CLIENTES

CREATE TABLE IF NOT EXISTS clientes (

    id BIGSERIAL PRIMARY KEY,

    nome VARCHAR(150) NOT NULL,

    email VARCHAR(120) NOT NULL UNIQUE,

    cpf VARCHAR(11) NOT NULL UNIQUE CHECK (cpf ~ '^\d{11}$'),

    telefone VARCHAR(20),

    endereco TEXT NOT NULL,

    cidade VARCHAR(100) NOT NULL,

    estado VARCHAR(2) NOT NULL,

    cep VARCHAR(8) NOT NULL,

    data_cadastro TIMESTAMP WITH TIME ZONE DEFAULT
CURRENT_TIMESTAMP,

    ativo BOOLEAN DEFAULT TRUE
```

```
);
```

```
CREATE INDEX idx_clientes_email ON clientes(email);
```

```
CREATE INDEX idx_clientes_cpf ON clientes(cpf);
```

```
-- TABELA: PEDIDOS
```

```
CREATE TABLE IF NOT EXISTS pedidos (
```

```
    id BIGSERIAL PRIMARY KEY,
```

```
    cliente_id BIGINT NOT NULL,
```

```
    data_pedido TIMESTAMP WITH TIME ZONE DEFAULT  
CURRENT_TIMESTAMP,
```

```
    status VARCHAR(20) NOT NULL DEFAULT 'pendente'
```

```
    CHECK (status IN ('pendente', 'confirmado', 'enviado',  
'entregue', 'cancelado')),
```

```
    total DECIMAL(10, 2) NOT NULL CHECK (total >= 0),
```

```
    desconto DECIMAL(10, 2) DEFAULT 0,
```

```
    frete DECIMAL(10, 2) DEFAULT 0,
```

```
CONSTRAINT fk_pedidos_cliente FOREIGN KEY (cliente_id)
```

```
REFERENCES clientes(id) ON DELETE CASCADE
```

```
);
```

```
CREATE INDEX idx_pedidos_cliente_id ON pedidos(cliente_id);
```

```
CREATE INDEX idx_pedidos_status ON pedidos(status);
```

```
-- TABELA: ITENS_PEDIDO
```

```
CREATE TABLE IF NOT EXISTS itens_pedido (
```

```
    id BIGSERIAL PRIMARY KEY,
```

```
    pedido_id BIGINT NOT NULL,
```

```
    produto_id BIGINT NOT NULL,
```

```
    nome_produto VARCHAR(255) NOT NULL,
```

```
    quantidade BIGINT NOT NULL CHECK (quantidade > 0),
```

```
    preco_unitario DECIMAL(10, 2) NOT NULL CHECK (preco_unitario  
>= 0),
```

```
    subtotal DECIMAL(10, 2) NOT NULL CHECK (subtotal >= 0),
```

```
    CONSTRAINT fk_itens_pedido FOREIGN KEY (pedido_id)
```

```
        REFERENCES pedidos(id) ON DELETE RESTRICT
```

```
);
```

```
CREATE INDEX idx_itens_pedido_pedido_id ON  
itens_pedido(pedido_id);
```

Queries Essenciais

SQL

```
-- Listar pedidos com dados do cliente
```

```
SELECT p.id, c.nome, p.data_pedido, p.status, p.total
```

```
FROM pedidos p
```

```
INNER JOIN clientes c ON p.cliente_id = c.id
```

```
ORDER BY p.data_pedido DESC;
```

```
-- Resumo de vendas por cliente
```

```
SELECT c.nome, COUNT(p.id) AS total_pedidos, SUM(p.total) AS  
valor_total
```

```
FROM clientes c
```

```
LEFT JOIN pedidos p ON c.id = p.cliente_id
```

```
GROUP BY c.id, c.nome
```

```
ORDER BY valor_total DESC;
```

Características de Qualidade

Item	Implementação
Integridade	FK com CASCADE/RESTRICT
Validação	CHECK constraints
Precisão	DECIMAL para valores monetários
Performance	Índices estratégicos
Segurança	UNIQUE em email/CPF
Escalabilidade	BIGSERIAL para IDs